MLFQ tries to optimize the turnaround time, and make the system responsive to interactive users.

THE CRUX:
HOW TO SCHEDULE WITHOUT PERFECT KNOWLEDGE?
How can we design a scheduler that both minimizes response time for interactive jobs while also minimizing turnaround time without *a priori* knowledge of job length?

Basic Rules:

↳ MLFQ has a number of distinct queues, each assigned a priority level.

- Rule 01: If Priority(A) > Priority(B), A runs (B doesn't)
- Rule 02: If Priority(A) = Priority(B), A & B run in RR

↳ MLFQ varies the priority of a job based on its observed behaviour.

↳ If a job is I/O intensive then it's priority will be high

↳ If a job is CPU intensive then it's priority will be low.

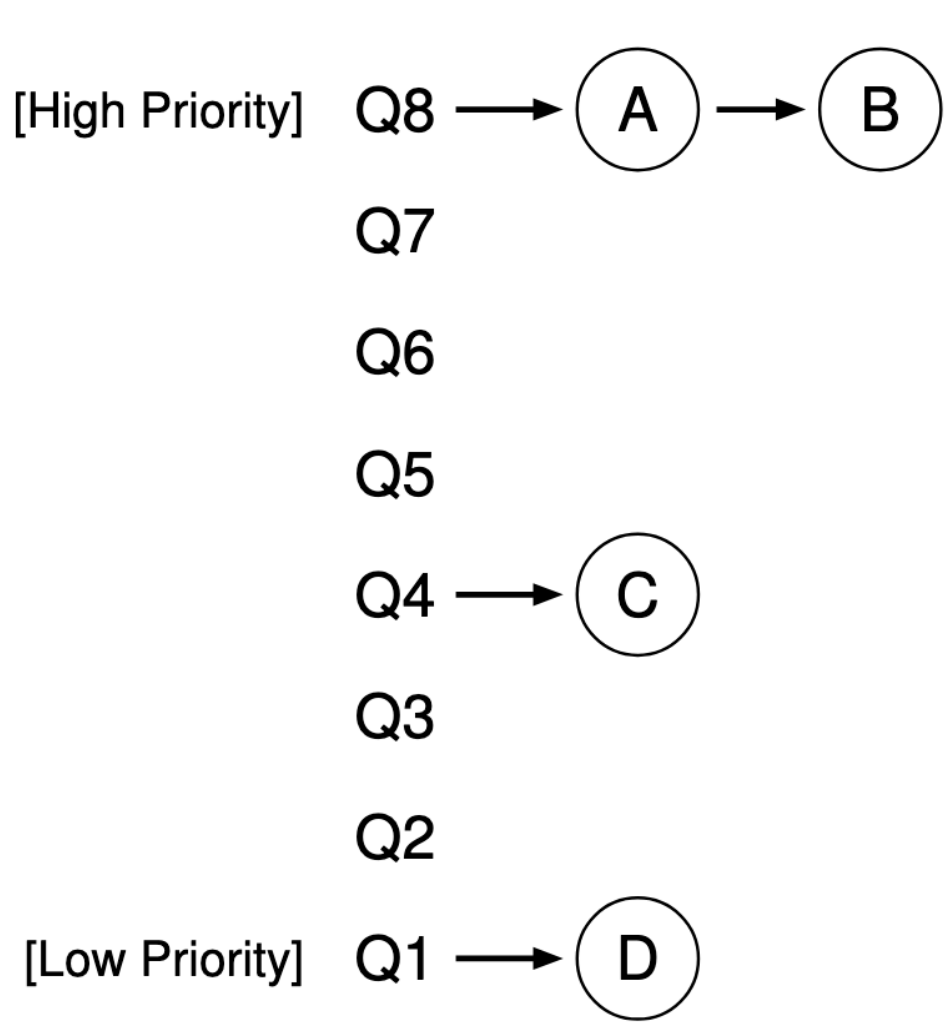


Figure 8.1: **MLFQ Example**

How to Change Priority

- How does MLFQ changes the priority of the job over it's lifetime?

- The allotement is the amount of time a job can spend at a given priority level before the scheduler reduces it's priority.

- For simplicity, allotement time = time slice

Rule 3: When a job enters the system, it is placed at the highest priority (the topmost queue).

Rule 4a: If a job uses up its allotment while running, its priority is *reduced* (i.e., it moves down one queue).

Rule 4b: If a job gives up the CPU (for example, by performing an I/O operation) before the allotment is up, it stays at the *same* priority level (i.e., its allotment is reset).

Example,

1. A single long running time

   - lets say allotement time = 10 ms.

   - At first the Job is at the highest priority.

   - After 10ms, the scheduler reduces it priority by one.

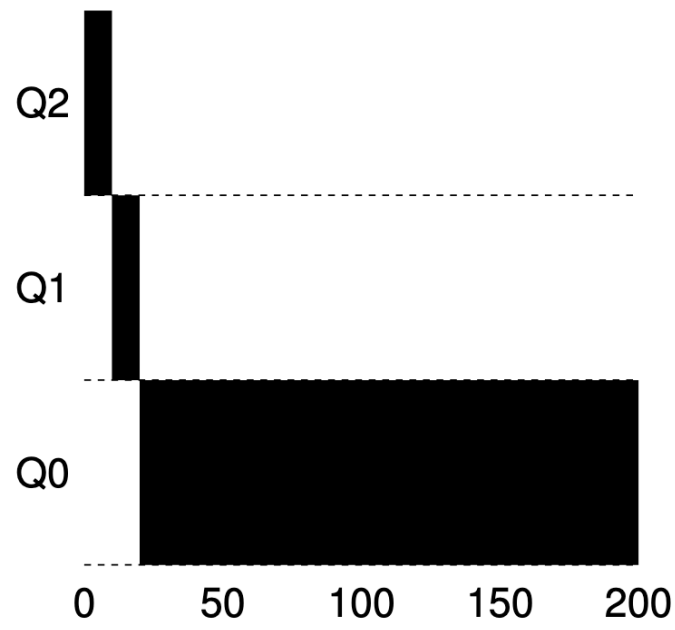


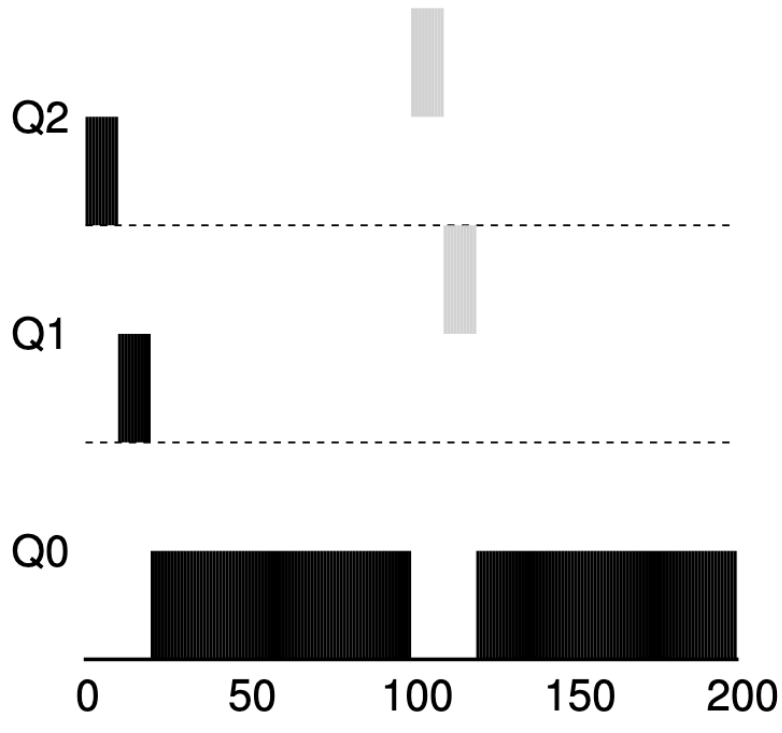
Figure 8.2: **Long-running Job Over Time**

   - Finally, it reaches the lowest level where it remains

2. Along came a short Job

   - In addition to a long running CPU intensive Job A, we have a short, interactive Job B.

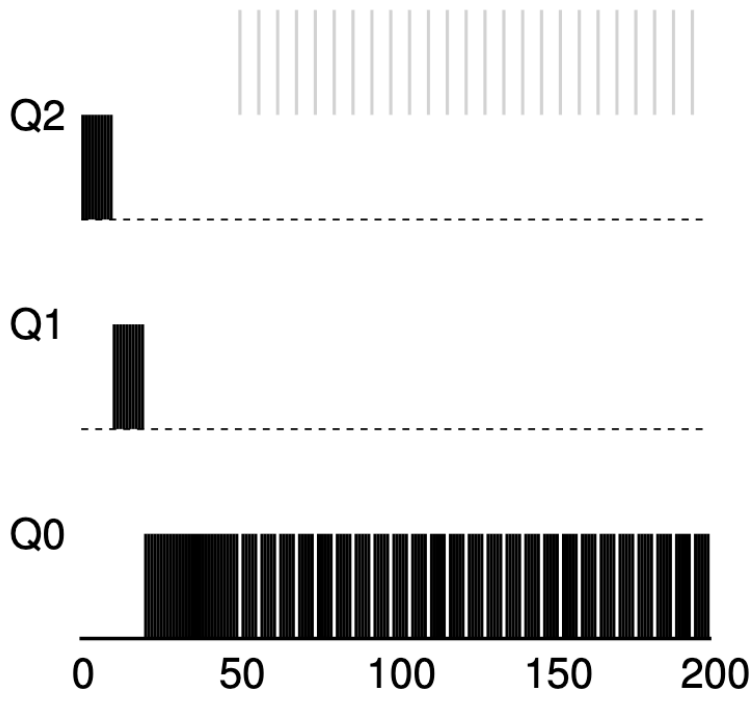
- After Job A falls to lowest priority level, at T=100, Job B arrives.
- It is inserted into the highest level, as it runs for 20 ms, Job B completes first
- In this manner, MLFQ approx SJF



- Because it doesn't know whether a job will be a short job or a long-running job, it first assumes it might be a short job, thus giving the job high priority. If it actually is a short job, it will run quickly and complete; if it is not a short job, it will slowly move down the queues, and thus soon prove itself to be a long-running more batch-like process.

3. What about I/O?

- Job B needs CPU for only 1 ms & Job A is long running & CPU intensive.
- MLFQ keeps Job B at the highest level



Problems with our current MLFQ

- Starvation:
  - If there are too many interactive jobs, then they will combine & consume all CPU time & long-running jobs will not get any CPU time.
- Game the schedules:
  - Refers to tricking the schedules into giving you more than your fair share of resources.
  - A job could issue I/O just before allotement

is used & thus remain at the highest level.

↳ A CPU intensive job might become interactive but schedulers learned it's priority to be low.

The Priority Boost

Rule 5: After some time period S, move all the jobs in the system to the topmost queue.

↳ This solves the starvation problem

↳ If a CPU intensive Job becomes interactive, the schedules will treat it properly

↳ We have Jobs A, B, C

↳ After every 100 ms, these is a priority boost & in this way long running jobs will make some progress

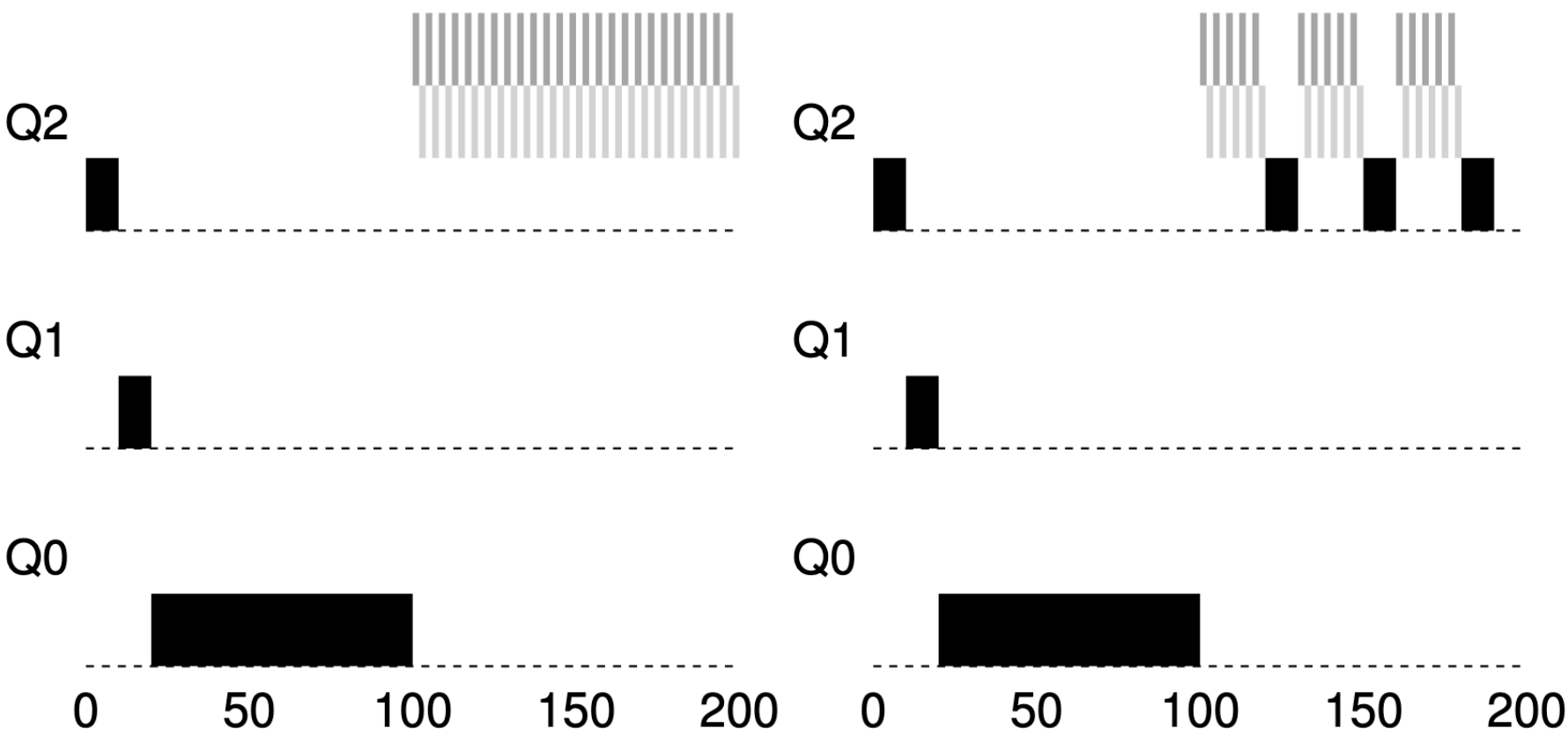


Figure 8.4: **Without (Left) and With (Right) Priority Boost**

↳ If S is too high, long running jobs could starve

↳ If S is too low, interactive may not get proper share of CPU.

Better accounting

↳ Rule 4a & 4b allow gamming the scheduler to get more

time on CPU.

So, we change it to:

- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).

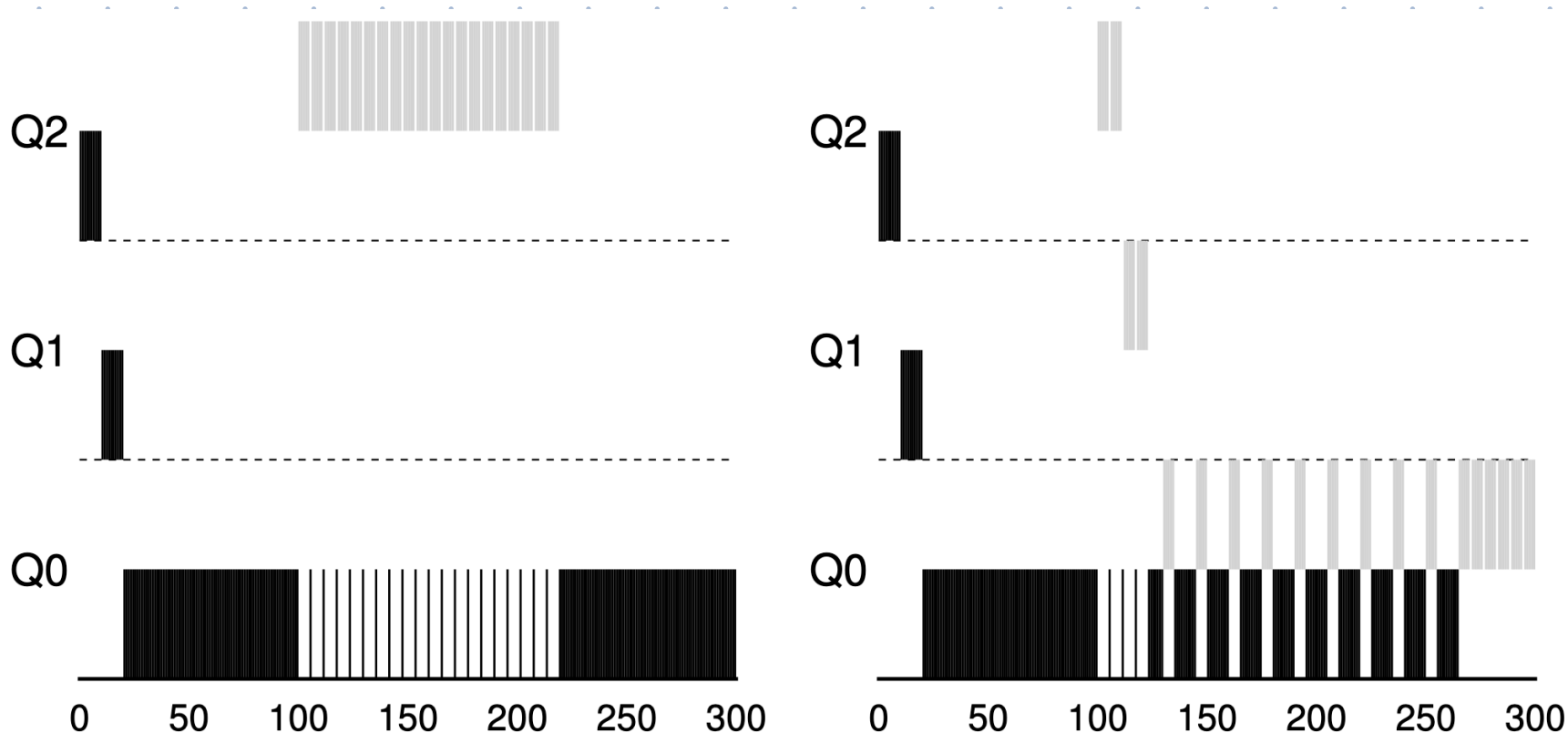


Figure 8.5: **Without (Left) and With (Right) Gaming Tolerance**

Tuning MLFQ & Other Issues:

How many queues?

How big should the time slice be per queue?

What should be value of S?

Most MLFQ variants give increasing time slices with decreasing priority.

Run for → 20 ms with 10 ms time slice

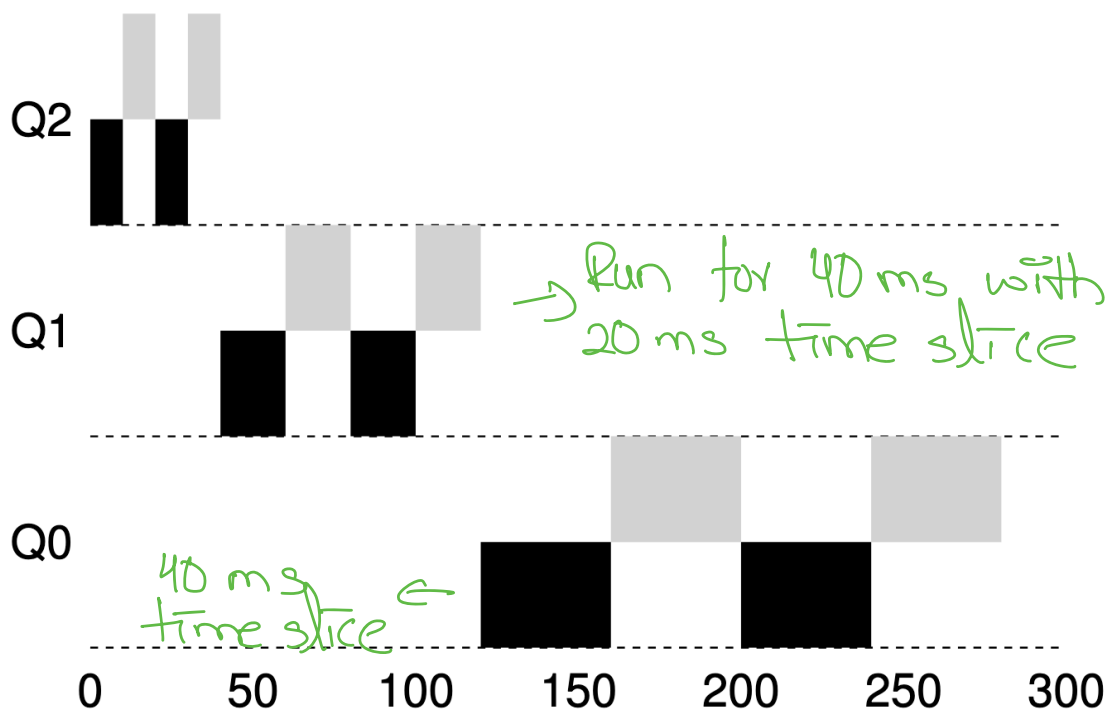


Figure 8.6: **Lower Priority, Longer Quanta**

- Solaris MLFQ implementation defines a configuration table.
- FreeBSD uses a formula to calculate the current priority of a job basing it on how much CPU the process has used.

  Usage is decayed overtime.
- Some reserve highest priority level for OS work.
- Some allow changing priority level using command line utility nice.

Summary

- **Rule 1:** If Priority(A) $>$ Priority(B), A runs (B doesn't).
- **Rule 2:** If Priority(A) $=$ Priority(B), A & B run in round-robin fashion using the time slice (quantum length) of the given queue.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.